Asignatura: Programación III

Tema: Proyecto Final

Participante: Samill Alexander Germoso Montero

Matrícula: 2022-1701

Facilitador: Kelyn Tejada Belliard

Tabla de contenido

Tabla de contenido *2*

1. Estrategia de Trabajo *3*

1.1 Nombre del Proyecto de Software 3

1.2 Tecnología para Aplicar 3

1.3 Objetivo del Proyecto 4

1.4 Alcance del Proyecto 4

1.5 Cronograma del Proyecto 5

1.6 Definición del Primer Release 5

2. Metodología Scrum *7*

2.1 Definición de Tareas a Ejecutar 7

2.2 Equipo de Trabajo 8

2.3 Herramientas 9

2.4 Épicas 10

2.5 Ceremonias de Scrum 10

2.6 Historias de Usuario 11

HU-002: Registro de Productos 11

3. Plan de Pruebas *15*

3.1 Lista de Requerimientos Funcionales y No Funcionales 15

3.2 Criterios de Aceptación y Rechazo de Pruebas 17

3.3 Herramientas de Pruebas 17

3.4 Cronograma de Ejecución de Pruebas 18

3.5 Plantillas para Casos de Prueba 18

3.6 Equipos de Pruebas y Responsabilidades 21

3.7 Plan de Automatización de Pruebas 22

3.8 Ejecución y Demostración 23

4. Demostración y Entregables *25*

4.1 Video Demostrativo 25

4.2 Links Funcionales 25

5. Conclusiones *26*

6. Bibliografía *27*

1. Estrategia de Trabajo

1.1 Nombre del Proyecto de Software

FerreSys — Sistema de Gestión de Inventario para Ferretería

1.2 Tecnología para Aplicar

Capa	Tecnología	Versión	Propósito
Backend	NestJS	11.x	Framework de servidor Node.js con arquitectura modular
ORM	Prisma	6.x	Mapeo objeto-relacional para interacción con base de datos
Base de Datos	SQLite	3.x	Base de datos relacional ligera en archivo local
Frontend	React	19.x	Librería de interfaz de usuario con componentes reutilizables
Bundler	Vite	7.x	Servidor de desarrollo y empaquetador de alta velocidad
Routing	React Router DOM	7.x	Navegación SPA sin recarga del navegador
HTTP Client	Axios	1.x	Peticiones HTTP al backend desde el frontend
PDF	PDFKit	0.17	Generación de reportes en formato PDF
Validación	class-validator	0.15	Validación de datos en el backend con decoradores
Lenguaje	TypeScript JavaScript	5.x ES2022	Tipado estático (backend) y dinámico (frontend)
Testing	Selenium WebDriver	4.x	Automatización de pruebas funcionales en el navegador
Test Runner	Mocha	10.x	Framework de ejecución de pruebas con reportes

Reportes	Mochawesome	7.x	Generación de reportes HTML de pruebas automatizadas
Control de Versiones	Git + GitHub	—	Versionamiento de código y colaboración

1.3 Objetivo del Proyecto

Desarrollar un sistema web completo de gestión de inventario para una ferretería que permita:

- **Registrar** y mantener un catálogo actualizado de productos con nombre, código único, precio, cantidad en stock y categoría.
- **Buscar** productos rápidamente por nombre o código para agilizar la atención al cliente.
- **Gestionar ventas** con un sistema de carrito que valide stock disponible, calcule totales automáticamente y descuente el inventario.
- **Generar reportes** de ventas filtrados por rango de fechas con descarga en formato PDF.
- **Automatizar pruebas** funcionales con Selenium para asegurar la calidad del software.

El sistema busca digitalizar los procesos manuales de una ferretería, reduciendo errores humanos en el control de inventario y la facturación.

1.4 Alcance del Proyecto

Dentro del Alcance

Módulo	Funcionalidades
Productos	CRUD completo (Crear, Leer, Actualizar, Eliminar)
Ventas	Registro de ventas con carrito de compras, validación de stock
Reportes	Generación de reportes por fechas, descarga en PDF
Dashboard	Panel con estadísticas en tiempo real
Pruebas	Suite completa de pruebas automatizadas con Selenium
API REST	Backend con endpoints documentados y validación de datos

Fuera del Alcance

- Autenticación y gestión de usuarios (login/roles)
- Integración con sistemas de pago electrónico
- Gestión de proveedores y órdenes de compra

- Aplicación móvil nativa
- Notificaciones por email/SMS
- Facturación fiscal (NCF/DGI)

1.5 Cronograma del Proyecto

Sprint	Período	Actividades	Responsable
Sprint 1	Semana 1-2	Configuración del entorno, diseño de BD, modelo de datos Prisma, setup de NestJS y React	Desarrollador
Sprint 2	Semana 3-4	Módulo de Productos (CRUD backend + frontend), Dashboard	Desarrollador
Sprint 3	Semana 5-6	Módulo de Ventas (carrito + registro), integración frontend-backend	Desarrollador
Sprint 4	Semana 7-8	Módulo de Reportes (filtro por fechas + PDF), pruebas automatizadas, documentación	Desarrollador QA

1.6 Definición del Primer Release

Release 1.0 — "MVP de Gestión de Inventario"

Descripción: Primera versión funcional del sistema que permite gestionar el inventario completo de la ferretería, registrar ventas y generar reportes.

Requerimientos Funcionales (RF)

ID	Requerimiento	Prioridad
RF-01	El sistema debe permitir registrar productos con nombre, código, precio, cantidad y categoría	Alta
RF-02	El sistema debe validar que el código de producto sea único	Alta
RF-03	El sistema debe permitir buscar productos por nombre o código	Alta
RF-04	El sistema debe mostrar la lista de todos los productos con indicadores de stock	Alta

RF-05	El sistema debe permitir eliminar productos del inventario	Media
RF-06	El sistema debe permitir crear ventas seleccionando productos y cantidades	Alta
RF-07	El sistema debe validar stock disponible antes de confirmar una venta	Alta
RF-08	El sistema debe descontar automáticamente el stock al registrar una venta	Alta
RF-09	El sistema debe calcular el total de la venta automáticamente	Alta
RF-10	El sistema debe generar reportes de ventas filtrados por rango de fechas	Media
RF-11	El sistema debe permitir descargar reportes de ventas en formato PDF	Media
RF-12	El sistema debe mostrar un dashboard con estadísticas generales	Baja

Requerimientos No Funcionales (RNF)

ID	**Requerimiento**	**Categoría**
RNF-01	El sistema debe responder a las peticiones en menos de 2 segundos	Rendimiento
RNF-02	La interfaz debe ser responsiva y funcional en pantallas ≥ 1024px	Usabilidad
RNF-03	El sistema debe validar datos de entrada tanto en frontend como en backend	Seguridad
RNF-04	El código debe seguir principios de arquitectura modular (NestJS modules)	Mantenibilidad
RNF-05	El sistema debe manejar errores gracefully con mensajes claros al usuario	Usabilidad

RNF-06	La base de datos SQLite debe soportar al menos 10,000 registros sin degradar rendimiento	Rendimiento
RNF-07	El frontend debe usar SPA (Single Page Application) para navegación sin recarga	Usabilidad
RNF-08	El sistema debe tener pruebas automatizadas con cobertura del 80%+ de funcionalidades	Calidad

2. Metodología Scrum

2.1 Definición de Tareas a Ejecutar

Backlog del Producto

ID	Tarea	Sprint	Estado
T-01	Configurar proyecto NestJS con Prisma y SQLite	Sprint 1	Completado
T-02	Diseñar e implementar esquema de BD (Product, Sale, SaleItem)	Sprint 1	Completado
T-03	Configurar proyecto React con Vite y React Router	Sprint 1	Completado
T-04	Implementar CRUD de productos en el backend (controller + service)	Sprint 2	Completado
T-05	Crear DTOs con validaciones (class-validator)	Sprint 2	Completado
T-06	Implementar página de registro de productos (frontend)	Sprint 2	Completado
T-07	Implementar página de búsqueda de productos (frontend)	Sprint 2	Completado
T-08	Implementar Dashboard con estadísticas	Sprint 2	Completado

T-09	Implementar Sidebar de navegación	Sprint 2	Completado
T-10	Implementar endpoint de creación de ventas con validación de stock	Sprint 3	Completado
T-11	Implementar carrito de compras en el frontend	Sprint 3	Completado
T-12	Implementar integración frontend-backend (Axios + proxy Vite)	Sprint 3	Completado
T-13	Implementar módulo de reportes (filtro por fechas)	Sprint 4	Completado
T-14	Implementar generación de PDF con PDFKit	Sprint 4	Completado
T-15	Crear suite de pruebas automatizadas con Selenium	Sprint 4	Completado
T-16	Documentación del proyecto	Sprint 4	Completado
T-17	Configurar repositorio GitHub y deploy	Sprint 4	Completado

2.2 Equipo de Trabajo

Rol	Nombre	Responsabilidades
Product Owner	Samill Germoso	Definir historias de usuario, priorizar backlog, aceptar incrementos
Scrum Master	Samill Germoso	Facilitar ceremonias, eliminar impedimentos, asegurar proceso Scrum
Desarrollador Full-Stack	Samill Germoso	Implementar backend (NestJS), frontend (React), base de datos
QA / Tester	Samill Germoso	Crear plan de pruebas, ejecutar pruebas manuales y automatizadas

Habilidades Requeridas

Habilidad	**Nivel**
TypeScript / JavaScript	Intermedio
NestJS (Node.js)	Intermedio
React (Hooks, useState, useEffect)	Intermedio
Prisma ORM	Básico-Intermedio
SQL (SQLite)	Básico
Selenium WebDriver	Básico
Git / GitHub	Intermedio
REST APIs	Intermedio

2.3 Herramientas

Herramienta	**Propósito**	**Justificación**
GitHub	Control de versiones y repositorio de código	Plataforma estándar de la industria, integración con CI/CD
Jira	Gestión de historias de usuario y sprints	Herramienta profesional para tablero Scrum y tracking
Visual Studio Code	Editor de código	Extensiones para TypeScript, React, Prisma y debugging
Postman	Pruebas de API REST	Facilita probar endpoints antes de integrar con frontend
Chrome DevTools	Debugging del frontend	Inspector de elementos, consola y red
SQLite Browser	Visualizar base de datos	Verificar datos directamente en el archivo DB

Selenium WebDriver	Automatización de pruebas	Pruebas funcionales end-to-end en navegador real

2.4 Épicas

Épica 1: Gestión de Inventario

Como ferretero, necesito un sistema para registrar y consultar el stock de productos para mantener control del inventario.

Historias incluidas: HU-001, HU-002, HU-003, HU-004

Épica 2: Sistema de Ventas

Como cajero, necesito un sistema de carrito de compras para registrar ventas de manera ágil y precisa.

Historias incluidas: HU-005, HU-006

Épica 3: Reportes y Análisis

Como gerente, necesito generar reportes de ventas por períodos para analizar el rendimiento del negocio.

Historias incluidas: HU-007, HU-008

Épica 4: Calidad y Testing

Como equipo de desarrollo, necesitamos pruebas automatizadas para asegurar que el sistema funcione correctamente.

Historias incluidas: HU-009, HU-010

2.5 Ceremonias de Scrum

Ceremonia	Frecuencia	Duración	Fecha Ejemplo	Participantes
Sprint Planning	Inicio de cada sprint	1 hora	Lunes de cada sprint	Todo el equipo
Daily Stand-up	Diario	15 minutos	Lunes a Viernes, 9:00 AM	Todo el equipo

Sprint Review	Fin de cada sprint	30 minutos	Viernes de cada sprint	Equipo + Stakeholders
Sprint Retrospective	Fin de cada sprint	30 minutos	Viernes de cada sprint (después del Review)	Todo el equipo
Backlog Refinement	Mitad de cada sprint	30 minutos	Miércoles de cada sprint	Product Owner + Dev

Calendario de Ceremonias (Sprint 4 — Ejemplo)

Fecha	**Ceremonia**	**Agenda**
Lunes - Semana 7	Sprint Planning	Planificar tareas de reportes y testing
Lun-Vie	Daily Stand-up	¿Qué hice ayer? ¿Qué haré hoy? ¿Impedimentos?
Miércoles - Semana 7	Refinement	Revisar y detallar historias pendientes
Viernes - Semana 8	Sprint Review	Demostrar reportes PDF y pruebas Selenium
Viernes - Semana 8	Retrospectiva	¿Qué salió bien, qué mejorar, acciones?

2.6 Historias de Usuario

HU-001: Visualización del Dashboard

Épica: Gestión de Inventario **Puntos de Historia:** 3

Como gerente de la ferretería, **quiero** ver un panel resumen con estadísticas del inventario y ventas, **para** tomar decisiones informadas sobre el negocio.

Criterios de Aceptación:

- El dashboard muestra 4 tarjetas: Productos Registrados, Ventas Realizadas, Ingresos Totales, Stock Bajo
- Los valores numéricos se actualizan en tiempo real al cargar la página
- Se muestra una tabla con las 5 ventas más recientes
- Los botones de acceso rápido navegan correctamente a cada sección

HU-002: Registro de Productos

Épica: Gestión de Inventario **Puntos de Historia:** 5

Como encargado de inventario, **quiero** registrar nuevos productos con toda su información, **para** mantener actualizado el catálogo de la ferretería.

Criterios de Aceptación:

- El formulario solicita: nombre, código, categoría, precio y cantidad
- Los campos obligatorios se validan antes de enviar (frontend y backend)
- El precio debe ser mayor a 0 y la cantidad no puede ser negativa
- No se permite registrar un producto con un código que ya existe
- Al registrar exitosamente, se muestra una alerta de éxito y el formulario se limpia
- El botón "Limpiar" resetea todos los campos del formulario

HU-003: Búsqueda de Productos

Épica: Gestión de Inventario **Puntos de Historia:** 3

Como vendedor de la ferretería, **quiero** buscar productos por nombre o código, **para** encontrar rápidamente lo que el cliente necesita.

Criterios de Aceptación:

- La página carga mostrando todos los productos existentes
- Se puede buscar por nombre (parcial) o por código
- Los resultados se muestran en una tabla con: Código, Nombre, Categoría, Precio, Stock
- El stock muestra badges de colores: verde (normal), amarillo (<10), rojo (0)
- Si no hay resultados, se muestra "Producto no encontrado"
- Se muestra un contador con la cantidad de productos encontrados

HU-004: Eliminación de Productos

Épica: Gestión de Inventario **Puntos de Historia:** 2

Como encargado de inventario, **quiero** eliminar productos descontinuados, **para** mantener limpio y actualizado el catálogo.

Criterios de Aceptación:

- Cada producto en la tabla tiene un botón "Eliminar"
- Al hacer clic en eliminar, se muestra una confirmación al usuario
- Si el usuario confirma, el producto se elimina y desaparece de la lista
- Se muestra una alerta de éxito indicando que el producto fue eliminado

HU-005: Registro de Ventas con Carrito

Épica: Sistema de Ventas **Puntos de Historia:** 8

Como cajero de la ferretería, **quiero** registrar ventas seleccionando productos y cantidades en un carrito, **para** llevar control preciso de las transacciones.

Criterios de Aceptación:

- El dropdown muestra solo productos con stock disponible (>0)
- Se puede seleccionar un producto, indicar cantidad y agregar al carrito
- El carrito muestra: nombre, código, precio unitario, cantidad, subtotal
- Los botones +/- permiten ajustar la cantidad sin exceder el stock
- El total se calcula automáticamente (suma de subtotales)
- Al finalizar la venta, el stock se descuenta del inventario
- El carrito se vacía después de una venta exitosa
- No se puede finalizar una venta con carrito vacío

HU-006: Validación de Stock en Ventas

Épica: Sistema de Ventas **Puntos de Historia:** 3

Como sistema, **quiero** validar que haya suficiente stock antes de confirmar una venta, **para** evitar ventas de productos que no están disponibles.

Criterios de Aceptación:

- Si la cantidad solicitada excede el stock, se muestra un error con la cantidad disponible
- El backend verifica el stock antes de procesar la venta (segunda validación)
- Si hay error de stock insuficiente, la venta no se procesa y no se descuenta inventario

HU-007: Generación de Reportes de Ventas

Épica: Reportes y Análisis **Puntos de Historia:** 5

Como gerente de la ferretería, **quiero** generar reportes de ventas filtrados por rango de fechas, **para** analizar el rendimiento del negocio en períodos específicos.

Criterios de Aceptación:

- El formulario permite seleccionar fecha de inicio y fecha de fin
- Se valida que ambas fechas estén presentes y que inicio < fin
- El reporte muestra: número de ventas, monto total, período, detalle por venta
- Cada venta muestra los productos vendidos con cantidades y subtotales
- Si no hay ventas en el período, se muestra un mensaje informativo

HU-008: Descarga de Reportes en PDF

Épica: Reportes y Análisis **Puntos de Historia:** 5

Como gerente de la ferretería, **quiero** descargar los reportes de ventas en formato PDF, **para** archivarlos o compartirlos con la administración.

Criterios de Aceptación:

- El botón "Descargar PDF" aparece después de generar un reporte
- El PDF contiene: encabezado con período, resumen total, tabla de ventas detallada
- El archivo se descarga con nombre descriptivo (ej: reporte_ventas_2026-01-01_2026-03-31.pdf)
- El PDF se genera en el backend usando PDFKit

HU-009: Pruebas Automatizadas del Sistema

Épica: Calidad y Testing **Puntos de Historia:** 8

Como desarrollador, **quiero** tener pruebas automatizadas para todos los flujos principales, **para** asegurar que el sistema funciona correctamente después de cada cambio.

Criterios de Aceptación:

- Se implementan pruebas con Selenium WebDriver
- Se cubren los 6 flujos principales: Dashboard, Crear Producto, Buscar, Eliminar, Ventas, Reportes
- Cada prueba incluye: camino feliz, pruebas negativas y pruebas de límites
- Se generan capturas de pantalla automáticas como evidencia
- Se genera un reporte HTML con Mochawesome

HU-010: Navegación y Usabilidad

Épica: Calidad y Testing **Puntos de Historia:** 2

Como usuario del sistema, **quiero** una interfaz intuitiva con navegación clara, **para** utilizar el sistema sin necesidad de capacitación

Criterios de Aceptación:

- El sidebar muestra enlaces a todas las secciones organizados por categoría
- La sección activa se resalta visualmente en el sidebar
- Las transiciones entre páginas son suaves (animación de entrada)
- Los mensajes de alerta (éxito/error) se muestran con colores diferenciados

Resumen de Puntos de Historia

Historia	Puntos	Sprint
HU-001: Dashboard	3	Sprint 2

HU-002: Registro de Productos	5	Sprint 2
HU-003: Búsqueda de Productos	3	Sprint 2
HU-004: Eliminación de Productos	2	Sprint 2
HU-005: Registro de Ventas	8	Sprint 3
HU-006: Validación de Stock	3	Sprint 3
HU-007: Reportes de Ventas	5	Sprint 4
HU-008: Descarga PDF	5	Sprint 4
HU-009: Pruebas Automatizadas	8	Sprint 4
HU-010: Navegación y Usabilidad	2	Sprint 2
Total	**44**	

Velocidad del equipo: ~11 puntos por sprint (2 semanas)

3. Plan de Pruebas

3.1 Lista de Requerimientos Funcionales y No Funcionales

Requerimientos Funcionales a Probar

ID	Requerimiento	Historia Relacionada	Tipo de Prueba
RF-01	Registrar producto con datos válidos	HU-002	Funcional, Automatizada
RF-02	Validar código de producto único	HU-002	Funcional, Automatizada
RF-03	Buscar producto por nombre	HU-003	Funcional, Automatizada
RF-04	Buscar producto por código	HU-003	Funcional, Automatizada

RF-05	Eliminar producto con confirmación	HU-004	Funcional, Automatizada
RF-06	Crear venta con carrito de productos	HU-005	Funcional, Automatizada
RF-07	Validar stock antes de vender	HU-006	Funcional, Automatizada
RF-08	Descontar stock automáticamente	HU-005	Funcional, Manual
RF-09	Calcular total de venta	HU-005	Funcional, Automatizada
RF-10	Generar reporte por rango de fechas	HU-007	Funcional, Automatizada
RF-11	Descargar reporte en PDF	HU-008	Funcional, Manual
RF-12	Mostrar dashboard con estadísticas	HU-001	Funcional, Automatizada

Requerimientos No Funcionales a Probar

ID	**Requerimiento**	**Método de Prueba**
RNF-01	Tiempo de respuesta < 2 segundos	Medición durante pruebas automatizadas
RNF-02	Interfaz responsiva ≥ 1024px	Prueba manual en diferentes resoluciones
RNF-03	Validación frontend y backend	Prueba automatizada (enviar datos inválidos)
RNF-04	Arquitectura modular	Revisión de código
RNF-05	Manejo de errores con mensajes claros	Prueba automatizada (provocar errores)

3.2 Criterios de Aceptación y Rechazo de Pruebas

Criterios de Aceptación

Criterio	Descripción
Funcionalidad	Todas las funcionalidades del primer Release funcionan según los criterios de las historias de usuario
Sin errores críticos	No existen errores que impidan el uso normal del sistema
Validación de datos	El sistema rechaza datos inválidos tanto en frontend como en backend
Navegación intuitiva	Un usuario puede navegar por todas las secciones sin confusión
Cobertura de pruebas	Al menos el 80% de los flujos principales tienen pruebas automatizadas

Criterios de Rechazo

Criterio	Descripción
Error en CRUD	Si alguna operación de crear, leer o eliminar productos falla
Error en ventas	Si el sistema permite vender productos sin stock
Cálculo incorrecto	Si el total de una venta no coincide con la suma de subtotales
Pérdida de datos	Si los datos no persisten después de recargar la página
Caída del sistema	Si el backend o frontend se detiene inesperadamente

3.3 Herramientas de Pruebas

Herramienta	Tipo	Justificación

Selenium WebDriver 4.x	Automatización E2E	Estándar de la industria para pruebas en navegador real. Soporta múltiples navegadores. Permite interactuar con elementos del DOM como un usuario real.
Mocha 10.x	Test Runner	Framework maduro y estable. Soporta pruebas asíncronas con async/await. Compatible con múltiples reporters. Permite organizar pruebas en describe/it.
Mochawesome 7.x	Reportes HTML	Genera reportes visualmente atractivos en HTML. Incluye gráficos de pastel con resultados. Permite incluir capturas de pantalla.
Chrome + ChromeDriver	Navegador de pruebas	Navegador más utilizado. ChromeDriver se integra perfectamente con Selenium. Soporte completo para DevTools Protocol.
Postman	Pruebas de API	Para verificar endpoints del backend de forma aislada antes de las pruebas E2E. Permite crear colecciones reutilizables.
Node.js Assert	Assertions	Módulo nativo de Node.js para aserciones. Sin dependencias adicionales. Simple y directo.

3.4 Cronograma de Ejecución de Pruebas

Fase	Período	Tipo de Prueba	Actividades
Fase 1	Sprint 2 - Semana 4	Manual	Pruebas de CRUD de productos, navegación, dashboard
Fase 2	Sprint 3 - Semana 6	Manual + API	Pruebas de ventas, validación de stock, integración
Fase 3	Sprint 4 - Semana 7	Automatización	Crear scripts de Selenium para todos los flujos
Fase 4	Sprint 4 - Semana 8	Ejecución Final	Ejecutar suite completa, generar reportes, documentar

3.5 Plantillas para Casos de Prueba

Plantilla Estándar

Campo	Descripción
ID del Caso	Identificador único (ej: HU002-CF-01)
Historia de Usuario	Historia relacionada (ej: HU-002)
Nombre	Título descriptivo del caso de prueba
Tipo	Camino Feliz / Prueba Negativa / Prueba de Límites
Precondiciones	Estado inicial requerido
Pasos	Secuencia de acciones a ejecutar
Datos de Prueba	Datos específicos utilizados
Resultado Esperado	Comportamiento esperado del sistema
Resultado Real	Comportamiento observado
Estado	✅ Pasó / ❌ Falló / ⚠️ Bloqueado
Evidencia	Captura de pantalla o referencia al archivo

Caso de Prueba Ejemplo: HU002-CF-02

Campo	Valor
ID	HU002-CF-02
Historia	HU-002: Registro de Productos
Nombre	Producto se registra exitosamente con datos válidos
Tipo	Camino Feliz
Precondiciones	El sistema está ejecutándose. No existe un producto con el código a utilizar.

Pasos	1. Navegar a /products/new 2. Escribir "Martillo Stanley 16oz" en nombre 3. Escribir código único en código 4. Seleccionar "Herramientas Manuales" en categoría 5. Escribir "850.00" en precio 6. Escribir "25" en cantidad 7. Hacer clic en "Guardar Producto"
Datos de Prueba	Nombre: Martillo Stanley 16oz, Código: HU2-XXXXX, Precio: 850.00, Qty: 25, Categoría: Herramientas Manuales
Resultado Esperado	Se muestra alerta "Producto registrado exitosamente". El formulario se limpia.
Resultado Real	[Se completa durante ejecución]
Estado	✅ Pasó
Evidencia	Screenshot: HU002_CF02_product_created_success.png

Caso de Prueba Ejemplo: HU002-PN-01

Campo	**Valor**
ID	HU002-PN-01
Historia	HU-002: Registro de Productos
Nombre	No permite registrar sin campos obligatorios
Tipo	Prueba Negativa
Precondiciones	El sistema está ejecutándose.
Pasos	1. Navegar a /products/new 2. No llenar ningún campo 3. Hacer clic en "Guardar Producto"
Datos de Prueba	Todos los campos vacíos
Resultado Esperado	Aparecen mensajes de error de validación debajo de los campos. El formulario no se envía.
Resultado Real	[Se completa durante ejecución]

Estado	✅ Pasó
Evidencia	Screenshot: HU002_PN01_empty_form_validation.png

Caso de Prueba Ejemplo: HU005-CF-06

Campo	**Valor**
ID	HU005-CF-06
Historia	HU-005: Registro de Ventas
Nombre	La venta se registra exitosamente
Tipo	Camino Feliz
Precondiciones	Existe al menos un producto con stock. El carrito tiene al menos un producto.
Pasos	1. Navegar a /sales/new 2. Seleccionar producto del dropdown 3. Establecer cantidad 4. Agregar al carrito 5. Hacer clic en "Finalizar Venta"
Datos de Prueba	Producto: Llave Ajustable 10", Cantidad: 4
Resultado Esperado	Se muestra alerta "Venta registrada exitosamente". Carrito se vacía. Stock se actualiza.
Resultado Real	[Se completa durante ejecución]
Estado	✅ Pasó
Evidencia	Screenshot: HU005_CF06_sale_registered.png

3.6 Equipos de Pruebas y Responsabilidades

Rol	**Responsable**	**Responsabilidades**

Líder de QA	Samill Germoso	Diseñar el plan de pruebas, definir estrategia, revisar resultados
Tester Manual	Samill Germoso	Ejecutar pruebas manuales, documentar resultados con capturas
Tester Automatización	Samill Germoso	Desarrollar scripts de Selenium, ejecutar suite automatizada
Analista de Resultados	Samill Germoso	Analizar reportes de Mochawesome, identificar defectos, reportar

3.7 Plan de Automatización de Pruebas

Estrategia de Automatización

Aspecto	Detalle
Tipo de Pruebas	End-to-End (E2E) — simulando acciones reales del usuario en el navegador
Herramienta	Selenium WebDriver 4.x con ChromeDriver
Lenguaje	JavaScript (Node.js)
Framework	Mocha con assertions nativas de Node.js
Reportes	Mochawesome (HTML con gráficos y capturas)
Cobertura objetivo	80%+ de los flujos funcionales principales

Estructura de los Tests

```
tests/
├── helpers.js                  # Utilidades compartidas
├── package.json                # Dependencias
├── screenshots/                # Capturas automáticas
├── reports/                    # Reportes HTML generados
└── tests/
    ├── hu001_dashboard.test.js       # 7 casos de prueba
    ├── hu002_crear_producto.test.js  # 8 casos de prueba
    ├── hu003_buscar_producto.test.js # 8 casos de prueba
```

```
├── hu004_eliminar_producto.test.js # 5 casos de prueba
├── hu005_crear_venta.test.js       # 9 casos de prueba
└── hu006_reporte_ventas.test.js    # 8 casos de prueba
```

Total: 45 casos de prueba automatizados divididos en:

- **Camino Feliz:** ~25 casos — Verificar que funcionalidades trabajan correctamente
- **Pruebas Negativas:** ~12 casos — Verificar manejo de errores y datos inválidos
- **Pruebas de Límites:** ~8 casos — Verificar comportamiento con valores extremos

Cómo Ejecutar las Pruebas

```
# 1. Asegurarse de que backend y frontend están corriendo
cd backend && npm run start:dev      # Puerto 3000
cd frontend && npm run dev           # Puerto 5173

# 2. Instalar dependencias de prueba
cd tests && npm install

# 3. Ejecutar todas las pruebas con reporte HTML
npm test

# 4. Ejecutar prueba específica
npm run test:hu2    # Solo HU-002

# 5. Ejecutar todas sin reporte HTML
npm run test:run
```

3.8 Ejecución y Demostración

Ver capturas de pantalla en la carpeta tests/screenshots/ y el reporte HTML en tests/reports/report.html.

Ejemplo de ejecución de prueba:

Proyecto Final — hu004_eliminar_producto.test.js

```
const assert = require('assert');
const {
  createDriver, takeScreenshot, navigateTo,
  waitFor, createTestProductAPI, deleteProductAPI, uniqueCode,
  By, until, TIMEOUT
} = require('../helpers');

describe('HU-004: Eliminación de Productos', function () {
  this.timeout(60000);
  let driver;
  let testProduct1, testProduct2;
  const code1 = uniqueCode('DEL1');
  const code2 = uniqueCode('DEL2');

  before(async function () {
```

```
germoso@10 tests %  npm run test:run

  HU-001: Visualización del Dashboard
    Camino Feliz
📸 Screenshot: hu001_cf01_dashboard_loaded_1776483917119.png
    ✔ HU001-CF-01: El dashboard carga correctamente (1305ms)
📸 Screenshot: hu001_cf02_stat_cards_visible_1776483917542.png
    1) HU001-CF-02: Se muestran las 4 tarjetas de estadísticas
📸 Screenshot: hu001_cf03_stat_values_numeric_1776483917877.png
    ✔ HU001-CF-03: Los valores de estadísticas son numéricos (324ms)
📸 Screenshot: hu001_cf04_quick_access_navigation_1776483918721.png
    ✔ HU001-CF-04: Los botones de acceso rápido funcionan (1399ms)
    Prueba Negativa
📸 Screenshot: hu001_pn01_no_errors_on_load_1776483931129.png
    ✔ HU001-PN-01: Dashboard no muestra errores con datos vacíos (11881ms)
    Navegación del Sidebar
📸 Screenshot: hu001_nav01_sidebar_sections_1776483931997.png
    ✔ HU001-NAV-01: El sidebar muestra todas las secciones (863ms)
📸 Screenshot: hu001_nav02_active_link_highlighted_1776483933168.png
    ✔ HU001-NAV-02: La sección activa se resalta en el sidebar (1240ms)

  HU-002: Registro de Productos
    Camino Feliz
📸 Screenshot: hu002_cf01_register_form_loaded_1776483935478.png
    ✔ HU002-CF-01: El formulario de registro carga correctamente (1419ms)
📸 Screenshot: hu002_cf02_form_filled_1776483936613.png
📸 Screenshot: hu002_cf02_product_created_success_1776483938950.png
    ✔ HU002-CF-02: Producto se registra exitosamente con datos válidos (3526ms)
📸 Screenshot: hu002_cf03_form_cleared_1776483939308.png
    ✔ HU002-CF-03: El formulario se limpia después de crear un producto (235ms)
    Prueba Negativa
📸 Screenshot: hu002_pn01_empty_form_validation_1776483940586.png
```

Proyecto Final — hu003_buscar_producto.test.js

```
const assert = require('assert');
const {
  createDriver, takeScreenshot, navigateTo,
  waitVisible, waitFor, clearAndType,
  createTestProductAPI, deleteProductAPI, uniqueCode,
  By, until, TIMEOUT
} = require('../helpers');

describe('HU-003: Búsqueda de Productos', function () {
  this.timeout(60000);
  let driver;
  let testProduct;
  const productCode = uniqueCode('BUS');
  const productName = 'Destornillador Phillips Test';

  before(async function () {
    driver = await createDriver();
    testProduct = await createTestProductAPI({
      name: productName,
      code: productCode,
      price: 350.00,
      quantity: 30,
      category: 'Herramientas Manuales'
    });
  });
```

```
germoso@10 tests %  npm run test:run

📸 Screenshot: hu001_nav01_sidebar_sections_1776483931997.png
    ✔ HU001-NAV-01: El sidebar muestra todas las secciones (863ms)
📸 Screenshot: hu001_nav02_active_link_highlighted_1776483933168.png
    ✔ HU001-NAV-02: La sección activa se resalta en el sidebar (1240ms)

  HU-002: Registro de Productos
    Camino Feliz
📸 Screenshot: hu002_cf01_register_form_loaded_1776483935478.png
    ✔ HU002-CF-01: El formulario de registro carga correctamente (1419ms)
📸 Screenshot: hu002_cf02_form_filled_1776483936613.png
📸 Screenshot: hu002_cf02_product_created_success_1776483938950.png
    ✔ HU002-CF-02: Producto se registra exitosamente con datos válidos (3526ms)
📸 Screenshot: hu002_cf03_form_cleared_1776483939308.png
    ✔ HU002-CF-03: El formulario se limpia después de crear un producto (235ms)
    Prueba Negativa
📸 Screenshot: hu002_pn01_empty_form_validation_1776483940586.png
    ✔ HU002-PN-01: No permite registrar sin campos obligatorios (1300ms)
^C
germoso@10 tests %
```

4. Demostración y Entregables

4.1 Video Demostrativo

https://youtu.be/nOrGkuwhdFk

El video muestra:

1. Dashboard con estadísticas generales
2. Registro de un nuevo producto (validaciones y éxito)
3. Búsqueda de productos por nombre y código
4. Eliminación de un producto
5. Creación de una venta completa con carrito
6. Generación de reporte de ventas y descarga de PDF
7. Ejecución de pruebas automatizadas con Selenium

4.2 Links Funcionales

Recurso	Link
Repositorio GitHub	https://github.com/Germoso/Proyecto-Final
Jira	https://mailonywilo.atlassian.net/jira/software/projects/PROYEC/boards/35/backlog
Código de Pruebas Automatizadas	https://github.com/Germoso/Proyecto-Final/tree/main/tests

5. Conclusiones

El desarrollo de **FerreSys** ha permitido aplicar de manera práctica los conceptos fundamentales de la Programación III y la metodología Ágil Scrum:

1. **Arquitectura de Software:** Se implementó una arquitectura cliente-servidor con separación clara entre frontend (React) y backend (NestJS), comunicados a través de una API REST.
2. **Metodología Scrum:** Se organizó el trabajo en 4 sprints con historias de usuario detalladas, criterios de aceptación claros y ceremonias planificadas, lo que permitió un desarrollo ordenado e incremental.
3. **Calidad del Software:** Se creó una suite completa de 45 pruebas automatizadas con Selenium WebDriver que cubren los flujos principales del sistema, incluyendo caminos felices, pruebas negativas y pruebas de límites.
4. **Herramientas Profesionales:** Se utilizaron herramientas estándar de la industria (Git, NestJS, Prisma, React, Selenium) que son ampliamente utilizadas en el desarrollo de software profesional.
5. **Aprendizaje Integral:** El proyecto integró conocimientos de bases de datos, desarrollo backend, desarrollo frontend, pruebas automatizadas y gestión de proyectos, proporcionando una experiencia completa del ciclo de vida del desarrollo de software.

6. Bibliografía

1. NestJS Documentation. (2026). *NestJS - A progressive Node.js framework*. https://docs.nestjs.com/
2. Prisma Documentation. (2026). *Prisma - Next-generation Node.js and TypeScript ORM*. https://www.prisma.io/docs/
3. React Documentation. (2026). *React - A JavaScript library for building user interfaces*. https://react.dev/
4. Vite Documentation. (2026). *Vite - Next Generation Frontend Tooling*. https://vitejs.dev/
5. Selenium Documentation. (2026). *Selenium WebDriver*. https://www.selenium.dev/documentation/webdriver/
6. Schwaber, K. & Sutherland, J. (2020). *La Guía de Scrum*. https://scrumguides.org/
7. Mozilla Developer Network. (2026). *MDN Web Docs*. https://developer.mozilla.org/
8. SQLite Documentation. (2026). *SQLite Home Page*. https://www.sqlite.org/docs.html